

TickerTaka

AI 기반 한국 주식 분석 플랫폼

1. 문제 정의 및 해결 방안

개인 투자자가 마주하는 정보 격차와 TickerTaka의 해결 방안

개인 투자자가 마주하는 정보 격차

편향된 정보

강세론 분석은 강세론만, 약세론은 약세론만 다루어 종목 하나에 대한 균형 잡힌 시각을 얻기 어렵습니다.

영향력 판단 불가

시장에 쏟아지는 수많은 뉴스나 공시가 실제 호재인지 악재인지 신속하게 판단하기 어렵습니다.

파편화된 기록

종목별로 진행된 분석 내역을 체계적으로 보관하고 나중에 쉽게 재참조할 수단이 부족합니다.

해결 방안

- **Bull·Bear AI** : 양방향 대립 토론으로 다각적 시각 확보
- **Moderator AI / Judge AI** : 독립적 검증으로 편향 제거
- 개인 투자자에게 **객관적 분석 리포트 자동 제공**

2. 핵심 기능 모듈

데이터 수집 & 분석

- **관심 종목 관리:** 종목 검색, 실시간 현재가 추적, 일괄 데이터 새로고침 기능.
- **감성 분석 (Sentiment):** 실시간 뉴스 및 공시에 대해 FinBERT 기반 호재/악재 1차 판정.
- **LLM 구조화:** Qwen2.5 모델을 통한 핵심 포인트, 리스크, 요약 정보의 구조화 보강.

멀티에이전트 & 연동

- **AI 토론 그래프:** LangGraph 기반 Bull, Bear, Moderator 에이전트 간의 자동화된 공방.
- **실시간 스트리밍:** SSE(Server-Sent Events)를 통해 발언이 한 카드씩 도착하는 UX 제공.
- **노션 자동 저장:** MCP 양방향 통신을 통해 토론 결과를 개인 Notion DB에 즉각 발행.

3. 시스템 아키텍처

Frontend: Next.js 16, React 19, Tailwind CSS.

Backend: FastAPI 비동기 서버 및 LangGraph.

Workers: FinBERT 및 Qwen 모델 비동기 처리.

Storage: PostgreSQL(메인 DB), Redis(캐싱/제어), ChromaDB(벡터 저장소).

Integration: MCP(Model Context Protocol) 기반 노션 연동.

3. AI 토론 그래프 (LangGraph)

병렬 데이터 수집: 주식 가격, 재무 정보, 실시간 뉴스 데이터를 `asyncio.gather`를 통해 빠르고 동시에 수집

의제 생성: 수집된 컨텍스트를 바탕으로 양측이 논의할 3개의 핵심 의제(Agenda)를 생성

에이전트 토론: 각 주제당 4턴으로 진행 (Bull 주장 → Bear 반박 → Bull 재반박 → Bear 재반박). 총 12번의 발언이 생성

실시간 검증: 에이전트의 매 발언 직후, 사회자가 개입하여 환각(Hallucination) 및 근거 없는 비약이 없는지 검증

최종 결론 노드: 사용자의 선택에 따라 중립적 요약(Summary) 또는 승패 판정(Judge) 노드로 라우팅되어 토론을 마무리

5. 감성 분석 파이프라인

1단계: 데이터 수집

외부 API(yfinance, DART, 뉴스)를 통해 시장 정보와 공시 자료를 실시간으로 가져옵니다.

2단계: FinBERT 분류

`snunlp/KR-FinBert-SC` 모델로 즉각적인 긍정/부정/중립 판정 및 영향도 스코어를 계산합니다.

3단계: Qwen 구조화

영향도가 높은 정보는 `Qwen2.5-Instruct`를 통과하여 핵심포인트와 리스크로 구조화 및 요약됩니다.

6. RAG — 하이브리드 검색 아키텍처

3

결합 검색 단계

정교한 컨텍스트 주입 방식

단순한 유사도 검색을 넘어, 의미론적 검색인 **벡터 검색 (ko-sroberta)**과 정확한 키워드 매칭인 **BM25**를 결합합니다.

두 결과를 **RRF (Reciprocal Rank Fusion)** 방식으로 병합하고, 선택적으로 **Reranker (bge-reranker-v2-m3)**를 통과시켜 가장 신뢰도 높은 Top-N 청크만을 토론 에이전트에게 제공합니다.

7. 결론 모드: 사회자 vs 판사

사회자 (Moderator) 모드

결론 톤: 중립적 요약

"양측 핵심 논거를 정리하면 다음과 같습니다."

- 양측의 주장을 한쪽으로 치우침 없이 객관적으로 요약 정리합니다.
- 판단을 독자에게 온전히 맡깁니다.
- 강도(승률) 막대나 면책 문구를 별도로 노출하지 않습니다.

판사 (Judge) 모드

결론 톤: 강세/약세 명확한 판정문

"근거를 종합할 때, 강세론 측 주장이 더 설득력 있다고 판단합니다."

- 근거(Evidence) 가중치를 기반으로 최종 승패 및 논리적 우위를 판정합니다.
- UI에 직관적인 Bull/Bear 강도 게이지 막대를 표시합니다.
- **필수 면책 문구:** 제공 정보는 단순 참고용임을 상단에 붉은 박스로 강조하여 표시합니다.

8. 노션 자동 저장 (MCP)

Model Context Protocol 연동

토론 결과를 **양방향 MCP** 통신으로 노션 지식 베이스에 즉시 확장합니다.

'노션에 저장' 클릭 → **낙관적 업데이트(Optimistic UI)**로 즉각 피드백 제공

백그라운드에서 MCP Client가 Notion DB 페이지 생성 → '노션에서 열기' 링크 자동 전환

9. 기술 스택

영역	도입 기술 및 도구
Frontend	Next.js 16, React 19, TypeScript, Tailwind CSS, Pretendard Font
Backend	FastAPI, SQLAlchemy 2, asyncpg, sse-starlette, Python MCP SDK
AI / LLM	LangGraph, LangChain, OpenAI GPT-4o-mini, FinBERT(snunlp), Qwen2.5
Vector & RAG	ChromaDB 1.0, sentence-transformers, rank-bm25, ko-sroberta
DB & Cache	PostgreSQL (Managed), Redis 7 (Throttle, Quote Cache)
Infra & Ops	Docker Compose, NCP GPU Node, Langfuse v4 (Trace), RAGAS

10. 역할 분담

허유찬 조장 · 백엔드/AI

- DB 수집 파이프라인
- RAG 고도화
- 양방향 MCP 구현
- vLLM 구현(Local GPU)
- 기타 보완사항(스트리밍, 도커 등)

이건희 AI / LLM 엔지니어

- 멀티에이전트 토론 그래프
- 멀티 프로바이더 + sLLM
- Langfuse 토론 Tracing
- RAGAS 평가 파이프라인

오흥찬 백엔드 · 인프라

- DB 인프라 구축·운영
- DART 공시 수집·파싱
- ChromaDB+BM25 RAG
- 감성분석 파이프라인
- 프론트 REST API

최진혁 프론트엔드

- 프론트엔드 전체 설계
- Next.js/React UI/UX
- SSE 실시간 스트리밍
- 감성분석 시각화
- 관심종목 UI

11. 평가 항목별 구현 (1)

Multi-Agent 구조: LangGraph Moderator(Supervisor) + Bull·Bear·Judge 서브 에이전트

에러 핸들링 · fallback: LLM retry/timeout + 그래프 데드라인 → fail-soft(세션 실패 · 락 해제 · SSE error)

sLLM + 검증 + langfuse: Qwen2.5-3B 감성분석 + moderator 검증 + langfuse trace

5대 설계문서: 유스케이스 · 컴포넌트 · 인터페이스 · 시퀀스 · ERD

Dockerise: Dockerfile + compose(app · Redis · Chroma · Ollama) 원커맨드

11. 평가 항목별 구현 (2)

MCP: Notion 저장 + Claude Desktop 연동

vLLM: 로컬 데스크톱 GPU 상시 서빙 + Tailscale로 노트북 연동 → 워커 OpenAI 호환 호출

정량 평가: RAGAS 3메트릭 + golden set + 회귀 테스트

RAG 고도화: Hybrid(BM25 + 벡터 + RRF) + Reranker

스트리밍·비동기: SSE 실시간 중계 + 비동기 워커

Thank You

TickerTaka : AI 기반 한국 주식 분석 플랫폼

 github.com/TickerTaka/TickerTaka-Frontend

 github.com/TickerTaka/TickerTaka-backend